Named Entity Recognition using BERT

Named Entity Recognition (NER) is a fundamental task in Natural Language Processing (NLP) that involves identifying and classifying entities such as person names, locations, organizations, and miscellaneous entities from text.

Installing the neccessary libraries

```
!pip install -q --upgrade transformers datasets accelerate evaluate seqeval
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 43.6/43.6 kB 3.2 MB/s eta 0:00:
  Preparing metadata (setup.py) ... done
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.4/10.4 MB 71.6 MB/s eta 0:00:0
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 529.0/529.0 kB 28.3 MB/s eta 0:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 6.2 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 48.9/48.9 MB 17.8 MB/s eta 0:00:0
  Building wheel for seqeval (setup.py) ... done
```

Importing the Libraries and checking the version

```
import transformers
import datasets
import accelerate
import numpy
import pandas

print("Transformers:", transformers.__version__)
print("Datasets:", datasets.__version__)
print("Accelerate:", accelerate.__version__)
print("NumPy:", numpy.__version__)
print("Pandas:", pandas.__version__)
```

```
Transformers: 5.6.2
Datasets: 4.8.5
Accelerate: 1.13.0
NumPy: 2.0.2
Pandas: 2.2.2
```

Loading the dataset

The CoNLL-2003 dataset is a widely used benchmark dataset for Named Entity Recognition (NER) tasks in Natural Language Processing. It was introduced as part of the Conference on Natural Language Learning 2003, with the objective of developing models capable of identifying and classifying named entities in text.

The dataset consists of newswire articles primarily sourced from Reuters and is available in English and German. It is specifically designed for sequence labeling tasks, where each word (token) in a sentence is assigned a corresponding entity tag.

The dataset includes four main types of named entities:

PER (Person): Names of individuals

ORG (Organization): Names of organizations or institutions

LOC (Location): Names of geographical locations

MISC (Miscellaneous): Other entities such as nationalities, events, etc.

```
def read_conll(file_path):
    sentences = []
    labels = []
    tokens = []
    ner_tags = []

    with open(file_path, "r", encoding="utf-8") as f:
        for line in f:
            if line.startswith("-DOCSTART-") or line.strip() == "":
                if tokens:
                    sentences.append(tokens)
                    labels.append(ner_tags)
                    tokens = []
                    ner_tags = []
            else:
                parts = line.strip().split()
                tokens.append(parts[0])
                ner_tags.append(parts[-1])

    return sentences, labels

train_tokens, train_labels = read_conll("/content/eng.train")
val_tokens, val_labels = read_conll("/content/eng.testa")
test_tokens, test_labels = read_conll("/content/eng.testb")

print(train_tokens[0])
print(train_labels[0])
```

```
['EU', 'rejects', 'German', 'call', 'to', 'boycott', 'British', 'lamb', '.']
['B-ORG', 'O', 'B-MISC', 'O', 'O', 'O', 'B-MISC', 'O', 'O']
```

A function is defined to read the CoNLL format data

```
from datasets import Dataset, DatasetDict

dataset = DatasetDict({
    "train": Dataset.from_dict({"tokens": train_tokens, "ner_tags": train_la
    "validation": Dataset.from_dict({"tokens": val_tokens, "ner_tags": val_l
```

```
    "test": Dataset.from_dict({"tokens": test_tokens, "ner_tags": test_label
})
```

```
dataset.shape
```

```
{'train': (14041, 2), 'validation': (3250, 2), 'test': (3453, 2)}
```

The dataset has three subset, Training set with 14041 samples, Validation set with 3250 samples, Test set with 3453 samples

```
print(dataset)
```

```
DatasetDict({
    train: Dataset({
        features: ['tokens', 'ner_tags'],
        num_rows: 14041
    })
    validation: Dataset({
        features: ['tokens', 'ner_tags'],
        num_rows: 3250
    })
    test: Dataset({
        features: ['tokens', 'ner_tags'],
        num_rows: 3453
    })
})
```

```
for row in dataset["train"]:
    print(row)
```

```
{'tokens': ['9-mth', 'Helibor', '3.73', 'pct', '3.70', 'pct'], 'ner_tags':
{'tokens': ['12-mth', 'Helibor', '3.89', 'pct', '3.87', 'pct'], 'ner_tags':
{'tokens': ['--', 'Helsinki', 'newsroom', '+358', '-', '0', '-', '680', '50
{'tokens': ['Barrick', 'gets', '93', 'pct', 'of', 'Arequipa', '.'], 'ner_tag
{'tokens': ['TORONTO', '1996-08-27'], 'ner_tags': ['B-LOC', 'O']}
{'tokens': ['Barrick', 'Gold', 'Corp', 'said', 'on', 'Tuesday', 'its', 'take
{'tokens': ['"', 'We', 'are', 'pleased', 'that', 'Arequipa', 'shareholders',
{'tokens': ['We', 'now', 'have', 'the', 'opportunity', 'to', 'realize', 'the
{'tokens': ['The', 'C$', '30-a-share', 'deal', 'means', 'Barrick', 'will',
{'tokens': ['Barrick', 'said', 'details', 'involving', 'the', 'allocation',
{'tokens': ['Barrick', "'s", 'offer', 'of', 'C$', '30', 'a', 'share', 'or',
{'tokens': ['Toronto-based', 'Barrick', ',', 'the', 'world', "'s", 'third',
{'tokens': ['Experts', 'have', 'speculated', 'the', 'deposit', 'has', 'poter
{'tokens': ['More', 'drilling', 'results', 'are', 'expected', 'soon', '.'],
{'tokens': ['The', 'Barrick', 'bid', 'took', 'observers', 'by', 'surprise',
{'tokens': ['Arequipa', 'shareholders', 'had', 'the', 'option', 'to', 'choos
{'tokens': ['Shares', 'were', 'to', 'be', 'pro-rated', 'if', 'more', 'than',
{'tokens': ['--', 'Reuters', 'Toronto', 'Bureau', '416', '941-8100'], 'ner_t
{'tokens': ['Penn', 'Treaty', 'terminates', 'acquisition', 'pact', '.'], 'ne
{'tokens': ['ALLENTOWN', ',', 'Pa', '.'], 'ner_tags': ['B-LOC', 'O', 'B-LOC
{'tokens': ['1996-08-27'], 'ner_tags': ['O']}
{'tokens': ['Penn', 'Treaty', 'American', 'Corp', 'said', 'Tuesday', 'it',
{'tokens': ['In', 'announcing', 'its', 'decision', ',', 'Penn', 'Treaty', 's
{'tokens': ['It', 'explained', 'the', '"', 'addition', 'of', 'a', 'New', 'Yo
{'tokens': ['--', 'New', 'York', 'Newsdesk', '212-859-1610', '.'], 'ner_tags
{'tokens': ['VNU', 'details', 'first-half', 'operating', 'profits', '.'], 'n
{'tokens': ['HAARLEM', ',', 'Netherlands', '1996-08-27'], 'ner_tags': ['B-LO
{'tokens': ['Publisher', 'VNU', 'gave', 'the', 'following', 'breakdown', 'of
{'tokens': ['H1', '1996', 'H1', '1995'], 'ner_tags': ['O', 'O', 'O', 'O']}
{'tokens': ['Sales', 'Op', 'profit', 'Sales', 'Op', 'profit'], 'ner_tags':
{'tokens': ['Consumer', 'magazines', '618', '90', '568', '80'], 'ner_tags':
{'tokens': ['Newspapers', '363', '49', '295', '46'], 'ner_tags': ['O', 'O',
```

```
import torch

device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f'Using device: {device}')
```

```
Using device: cpu
```

Named Entity Recognition (NER) assigns a label to each word to show if it is part of an entity. “O” means not an entity. “B-” marks the beginning of an entity, and “I-” marks its continuation. Common types include PER (person), ORG (organization), LOC (location), and MISC (other entities like nationalities or events).

EXAMPLE

"Indian scientist APJ Abdul Kalam worked at ISRO in India"

Indian -> B-MISC

scientist -> O

APJ -> B-PER

Abdul -> I-PER

Kalam -> I-PER

worked -> O

at -> O

ISRO -> B-ORG

in -> O

India -> B-LOC

Create label mapping

```
unique_labels = set(label for doc in train_labels for label in doc)

label_list = sorted(list(unique_labels))
label2id = {label: i for i, label in enumerate(label_list)}
id2label = {i: label for label, i in label2id.items()}
```

Encode labels

```
def encode_labels(example):
    example["ner_tags"] = [label2id[label] for label in example["ner_tags"]]
    return example

dataset = dataset.map(encode_labels)
```

```
Map: 100% 14041/14041 [00:04<00:00, 2536.72 examples/s]
Map: 100% 3250/3250 [00:01<00:00, 3080.97 examples/s]
Map: 100% 3453/3453 [00:01<00:00, 3116.20 examples/s]
```

Converting textual NER labels into numerical format, which is required for training the model.

In the dataset, named entity tags such as PER, ORG, LOC, and MISC are initially stored as text labels. However, machine learning models like BERT cannot process textual labels directly. Therefore, these labels must be transformed into integer IDs.

Tokenizer

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: Us
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings ta
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access p
  warnings.warn(
config.json: 100%                                   570/570 [00:00<00:00, 9.41kB/s]
tokenizer_config.json: 100%                         49.0/49.0 [00:00<00:00, 1.98kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
vocab.txt:        213k/? [00:00<00:00, 1.88MB/s]
tokenizer.json:        436k/? [00:00<00:00, 4.14MB/s]
```

A tokenizer converts text into numbers that a model can understand.

"Hello world" → ["Hello", "world"] → [101, 7592, 2088, 102]

The tokenizer used here is bert-base-cased, a pre-trained tokenizer corresponding to the BERT model that preserves the case (uppercase and lowercase) of words.

```
text = "John lives in New York" # Example for testing

tokens = tokenizer(text)
print(tokens)
```

```
{'input_ids': [101, 1287, 2491, 1107, 1203, 1365, 102], 'token_type_ids': [0,
```

input_ids → numbers

attention_mask → which tokens matter

```
def tokenize_and_align_labels(examples):
    tokenized_inputs = tokenizer(
        examples["tokens"],
        truncation=True,
        padding=True,
        is_split_into_words=True
    )

    labels = []
    for i, label in enumerate(examples["ner_tags"]):
        word_ids = tokenized_inputs.word_ids(batch_index=i)
        previous_word_idx = None
        label_ids = []

        for word_idx in word_ids:
            if word_idx is None:
```

```
                label_ids.append(-100)
            elif word_idx != previous_word_idx:
                label_ids.append(label[word_idx])
            else:
                label_ids.append(-100)

            previous_word_idx = word_idx

        labels.append(label_ids)

    tokenized_inputs["labels"] = labels
    return tokenized_inputs
```

This function tokenizes input words and aligns their NER labels so they match the subword tokens used by models like BERT. It converts each word into tokens, maps those tokens back to their original words, assigns the correct label only to the first token of each word, and marks special or subword tokens with -100 so they are ignored during training. This ensures accurate label alignment and prepares the data for effective model training.

```
tokenized_datasets = dataset.map(tokenize_and_align_labels,batched=True)
```

```
Map: 100% 14041/14041 [00:07<00:00, 1590.51 examples/s]
Map: 100% 3250/3250 [00:03<00:00, 728.92 examples/s]
Map: 100% 3453/3453 [00:02<00:00, 1435.25 examples/s]
```

```
print(tokenized_datasets)
```

```
DatasetDict({
    train: Dataset({
        features: ['tokens', 'ner_tags', 'input_ids', 'token_type_ids', 'atte
        num_rows: 14041
    })
    validation: Dataset({
        features: ['tokens', 'ner_tags', 'input_ids', 'token_type_ids', 'atte
        num_rows: 3250
    })
    test: Dataset({
        features: ['tokens', 'ner_tags', 'input_ids', 'token_type_ids', 'atte
        num_rows: 3453
    })
})
```

Loading the Model

```
from transformers import AutoModelForTokenClassification

model = AutoModelForTokenClassification.from_pretrained(
```

```
    "bert-base-cased",
    num_labels=len(label_list),
    id2label=id2label,
    label2id=label2id
)
```

```
model.safetensors: 100%                                  436M/436M [00:07<00:00, 94.6MB/s]

Loading weights: 100%                                    197/197 [00:00<00:00,  9.62it/s]

[transformers] BertForTokenClassification LOAD REPORT from: bert-base-cased
Key                                          | Status     |
---------------------------------------------+------------+-
bert.pooler.dense.bias                       | UNEXPECTED |
cls.seq_relationship.bias                    | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight   | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias     | UNEXPECTED |
cls.predictions.transform.dense.bias         | UNEXPECTED |
bert.pooler.dense.weight                     | UNEXPECTED |
cls.predictions.transform.dense.weight       | UNEXPECTED |
cls.seq_relationship.weight                  | UNEXPECTED |
cls.predictions.bias                         | UNEXPECTED |
classifier.weight                            | MISSING    |
classifier.bias                              | MISSING    |

Notes:
- UNEXPECTED:    can be ignored when loading from different task/architecture;
- MISSING:  those params were newly initialized because missing from the chec
```

This code loads a BERT model. The parameter num_labels=len(label_list) tells the model how many different entity categories it needs to predict (like PER, LOC, ORG, etc.). The dictionaries id2label and label2id are used to map between numerical outputs (like 0,1,2) and actual entity names (like "B-PER", "I-LOC"), ensuring the predictions are interpretable.

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="./bert_ner_model",
    logging_dir="./logs",
    logging_steps=50,
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    weight_decay=0.01,
    num_train_epochs=1,
    max_steps=1000,
    eval_strategy="steps",
    eval_steps=200,
    save_steps=200,
    save_strategy="epoch",
    report_to="none"
```

```
)
```

```
[transformers] `logging_dir` is deprecated and will be removed in v5.2. Pleas
```

This code sets the training configuration for the model using Hugging Face. It defines where to save the model, how often to log progress, and controls learning with a small learning rate. The batch size (32) decides how many samples are processed at once. Training is limited to 1 epoch and 500 steps to reduce time. It also evaluates and saves the model after each epoch, while disabling external logging.

```
import evaluate

metric = evaluate.load("seqeval")

def compute_metrics(p):
    predictions, labels = p
    predictions = predictions.argmax(axis=2)

    true_predictions = [
        [id2label[p] for (p, l) in zip(pred, lab) if l != -100]
        for pred, lab in zip(predictions, labels)
    ]

    true_labels = [
        [id2label[l] for (p, l) in zip(pred, lab) if l != -100]
        for pred, lab in zip(predictions, labels)
    ]

    results = metric.compute(
        predictions=true_predictions,
        references=true_labels)

    return {
        "precision": results["overall_precision"],
        "recall": results["overall_recall"],
        "f1": results["overall_f1"],
        "accuracy": results["overall_accuracy"],
    }
```

```
Downloading builder script:        6.34k/? [00:00<00:00, 163kB/s]
```

```
from transformers import DataCollatorForTokenClassification

data_collator = DataCollatorForTokenClassification(tokenizer)
```

This code creates a data collator using the Hugging Face tokenizer. It automatically pads sequences in each batch to the same length and keeps tokens and labels

aligned during training.

```
from transformers import Trainer

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
    processing_class=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics)
```

```
train_result = trainer.train()
train_result
```

[1000/1000 5:05:05, Epoch 0/1]

Step	Training Loss	Validation Loss	Precision	Recall	F1	Accuracy
200	0.051901	0.088817	0.863273	0.890441	0.876647	0.979615
400	0.077590	0.069105	0.889072	0.906429	0.897667	0.982808
600	0.063814	0.060854	0.903732	0.921070	0.912319	0.985573
800	0.059444	0.055713	0.918618	0.930831	0.924684	0.987170
1000	0.059366	0.053295	0.922730	0.930495	0.926596	0.987462

[201/1000 40:55 < 2:44:19, 0.08 it/s, Epoch 0.11/1]

Step	Training Loss	Validation Loss

[407/407 1:03:54]

Writing model shards: 100% 1/1 [00:03<00:00, 3.84s/it]

```
TrainOutput(global_step=1000, training_loss=0.07126196765899659, metrics=
{'train_runtime': 18354.5405, 'train_samples_per_second': 0.436,
'train_steps_per_second': 0.054, 'total_flos': 508629993350976.0,
'train_loss': 0.07126196765899659, 'epoch': 0.5694760820045558})
```

The model has a final accuracy of 0.9865 with a precision of 0.91 and recall of 0.92 which shows the model is correctly identifying the entities.

The Validation loss and training loss is decreasing over each steps which indicates that the model is learninh effectively.

```
def calibrate_confidence(results, temperature=1.5):
    for r in results:
        r['score'] = float(np.exp(np.log(r['score']) / temperature))
    return results
```

```
trainer.save_model("./ner_model")
tokenizer.save_pretrained("./ner_model")

print("Model saved successfully.")
```

```
Writing model shards: 100%                                  1/1 [00:05<00:00,  5.28s/it]
Model saved successfully.
```

```
from transformers import pipeline

ner_pipeline = pipeline(
    "ner",
    model="./ner_model",
    tokenizer="./ner_model"
)

print("Pipeline ready.")
```

```
Loading weights: 100%                                  199/199 [00:00<00:00, 3013.08it/s]
Pipeline ready.
```

Testing the model with random inputs

Example 1

```
text1 = "Barack Obama was born in Hawaii and worked at Microsoft."

results = ner_pipeline(text1)

for r in results:
    word = r['word']
    tag = r['entity']

    if "PER" in tag:
        print(f"{word} → Person")
    elif "ORG" in tag:
        print(f"{word} → Organization")
    elif "LOC" in tag:
        print(f"{word} → Location")
    elif "MISC" in tag:
        print(f"{word} → Miscellaneous")
    else:
        print(f"{word} → Other")
```

```
Barack → Person
Obama → Person
Hawaii → Location
Microsoft → Organization
```

Example2

```
text2 = "She completed her degree at Stanford University in California."
results = ner_pipeline(text2)

for r in results:
    word = r['word']
    tag = r['entity']

    if "PER" in tag:
        print(f"{word} → Person")
    elif "ORG" in tag:
        print(f"{word} → Organization")
    elif "LOC" in tag:
        print(f"{word} → Location")
    elif "MISC" in tag:
        print(f"{word} → Miscellaneous")
    else:
        print(f"{word} → Other")
```

```
Stanford → Organization
University → Organization
California → Location
```

Example3

```
text3 = " Tesla opened a new factory near Berlin last year."
results = ner_pipeline(text3)

for r in results:
    word = r['word']
    tag = r['entity']

    if "PER" in tag:
        print(f"{word} → Person")
    elif "ORG" in tag:
        print(f"{word} → Organization")
    elif "LOC" in tag:
        print(f"{word} → Location")
    elif "MISC" in tag:
        print(f"{word} → Miscellaneous")
    else:
        print(f"{word} → Other")
```

```
Te → Organization
##sla → Organization
Berlin → Location
```

The model is efficiently identifying the entities

```
user_text = input("Enter a sentence: ")
results = ner_pipeline(user_text)
print(results)
```

```
Enter a sentence: Mahatma Gandi is Father of India
[{'entity': 'B-PER', 'score': np.float32(0.9843592), 'index': 1, 'word': 'Ma'
```

Evaluation

```
from sklearn.metrics import classification_report

predictions, labels, _ = trainer.predict(tokenized_datasets["test"])
preds = predictions.argmax(axis=2)

true_labels = []
true_preds = []

for pred, label in zip(preds, labels):
    for p, l in zip(pred, label):
        if l != -100:
            true_labels.append(l)
            true_preds.append(p)

print(classification_report(true_labels, true_preds))
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: U
  super().__init__(loader)
              precision    recall  f1-score   support

           0       0.92      0.93      0.93      1668
           1       0.80      0.80      0.80       702
           2       0.90      0.89      0.90      1661
           3       0.96      0.96      0.96      1617
           4       0.84      0.89      0.86       257
           5       0.62      0.67      0.64       216
           6       0.88      0.91      0.89       835
           7       0.98      0.99      0.99      1156
           8       1.00      0.99      0.99     38323

    accuracy                           0.98     46435
   macro avg       0.88      0.89      0.89     46435
weighted avg       0.98      0.98      0.98     46435
```

The overall accuracy is 0.98 (98%), indicating that the model performs very well on the test dataset.

The weighted average F1-score is 0.98, which means the model is highly effective when considering class imbalance.

The macro average F1-score is 0.89, which is slightly lower, suggesting that performance varies across classes.

```
Start coding or generate with AI.
```